

Decision graph for the card-servicing agent

Every node, branch and loop from member message to audited terminal state — drawn as one spine and five phases rather than a single wall. The LLM handles language only; confidence, policy outcome, autonomy tier and execution authority are decided by deterministic code.

12 stages 41 nodes 11 loops 7 outcomes 5 retrieval paths

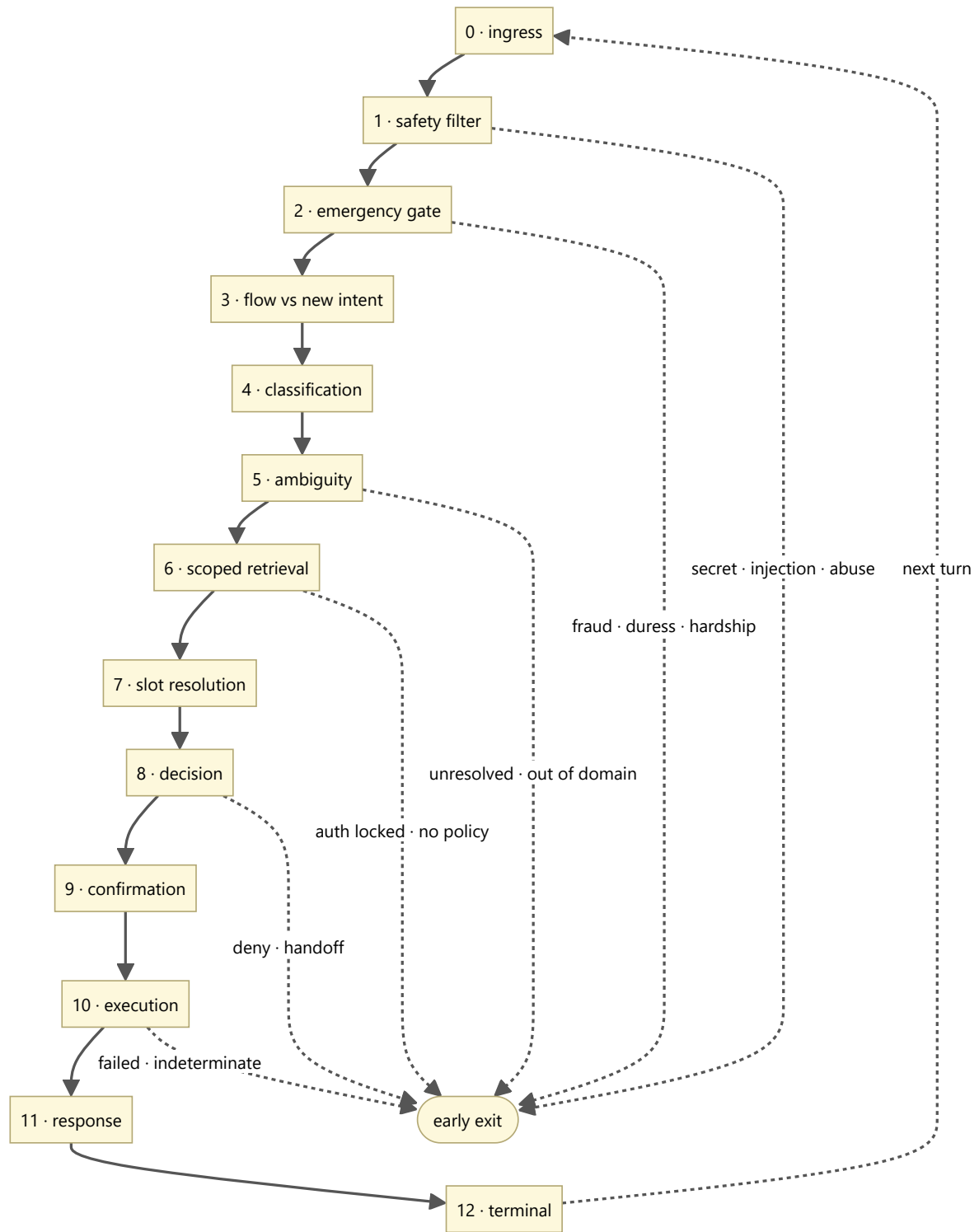
00	01	02	03	04	05	06	07	08	09	10	11	12
Ingress	Safety filter	Emergency	Flow vs intent	Classify	Ambiguity	Retrieval	Slots	Decision	Confirm	Execution	Response	Terminal

rectangle = process step diamond = decision gate stadium = entry or exit solid arrow = forward
dotted arrow = loop-back

01 The spine

all stages

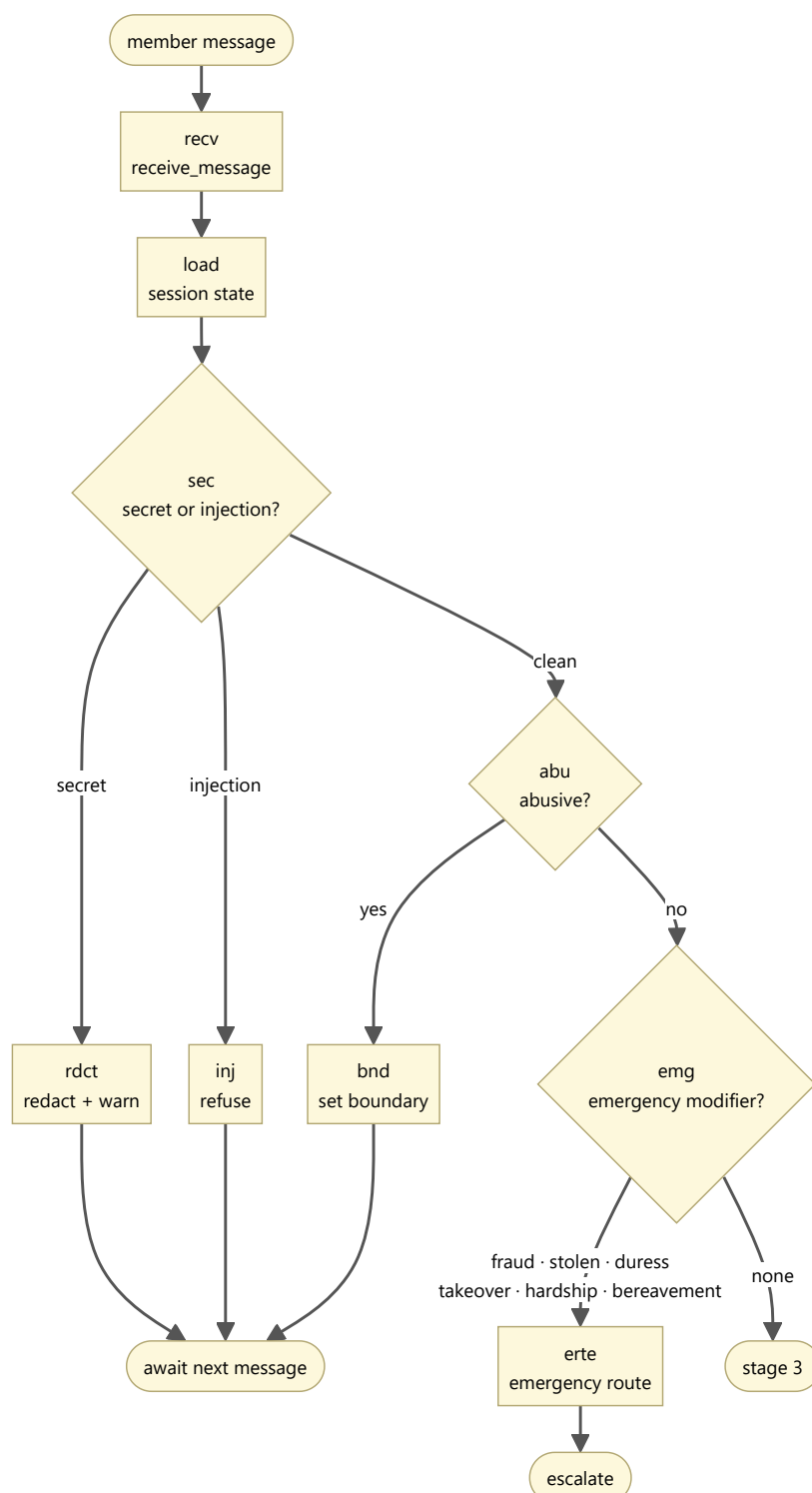
The happy path runs straight down. Six stages can exit early to a terminal state — that early-exit density is the point of the design, not a flaw in it.



02 Phase A — gates

stages 0-2

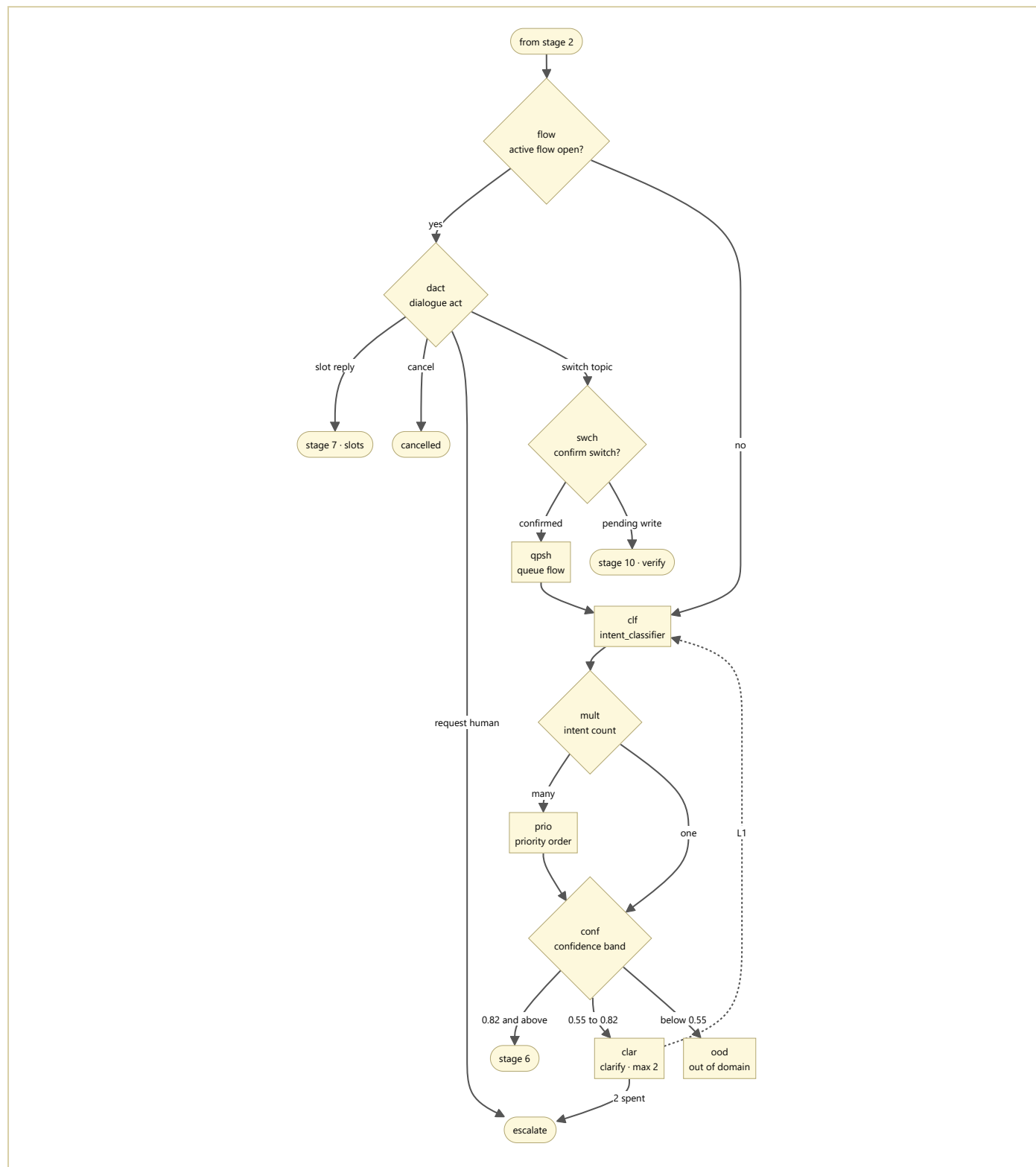
Two filters run before the model is called at all. A PIN in the message must never reach the LLM, so redaction cannot sit downstream of classification. The emergency gate is second because "waive my fee, and I also see a charge I never made" has to service the fraud first.



03 Phase B — understanding

stages 3-5

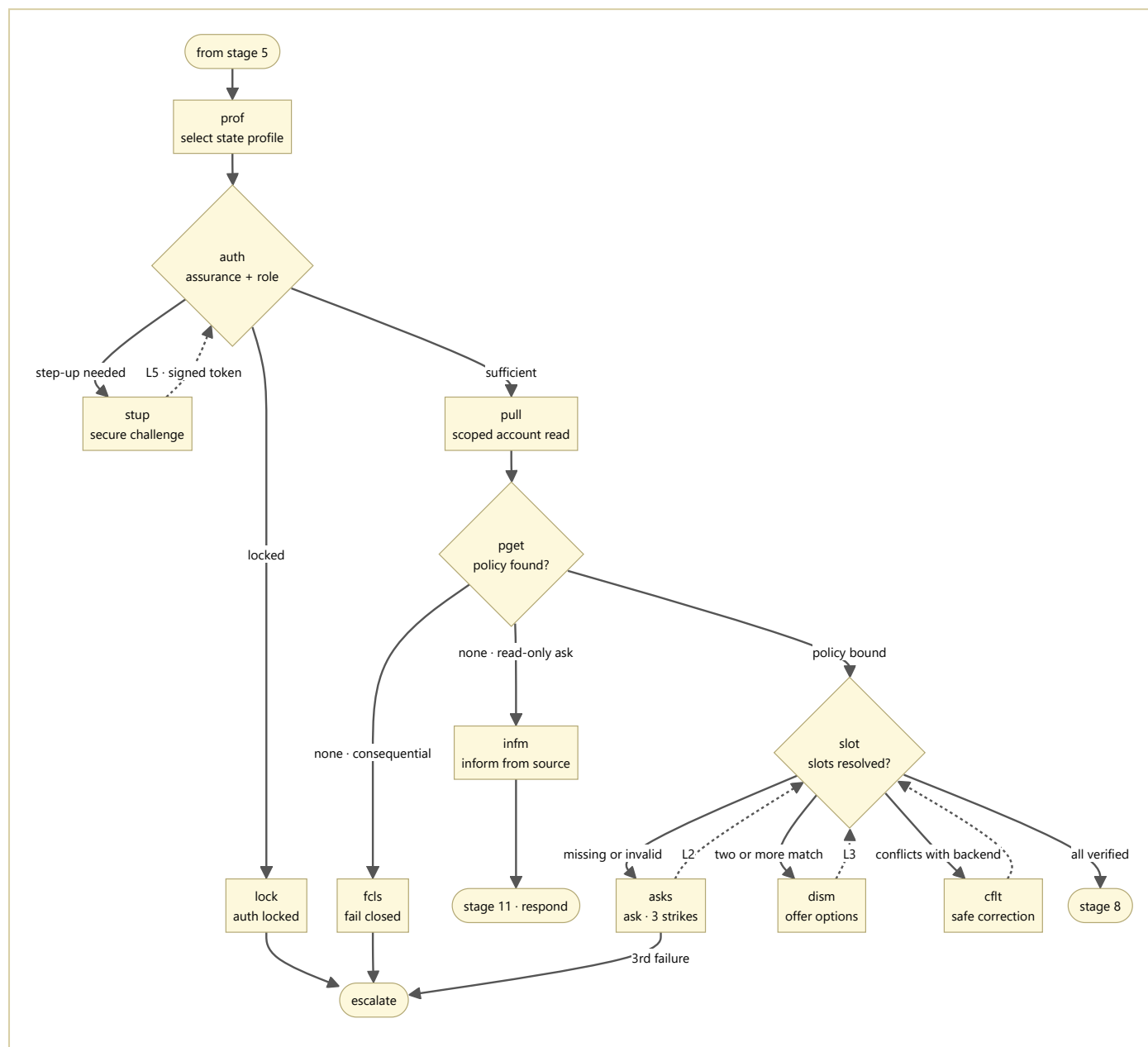
The dialogue-act check comes before reclassification. That is what makes "the second one" fill a slot instead of starting a new request — the single most common way a servicing bot loses the thread.



04 Phase C — grounding

stages 6-7

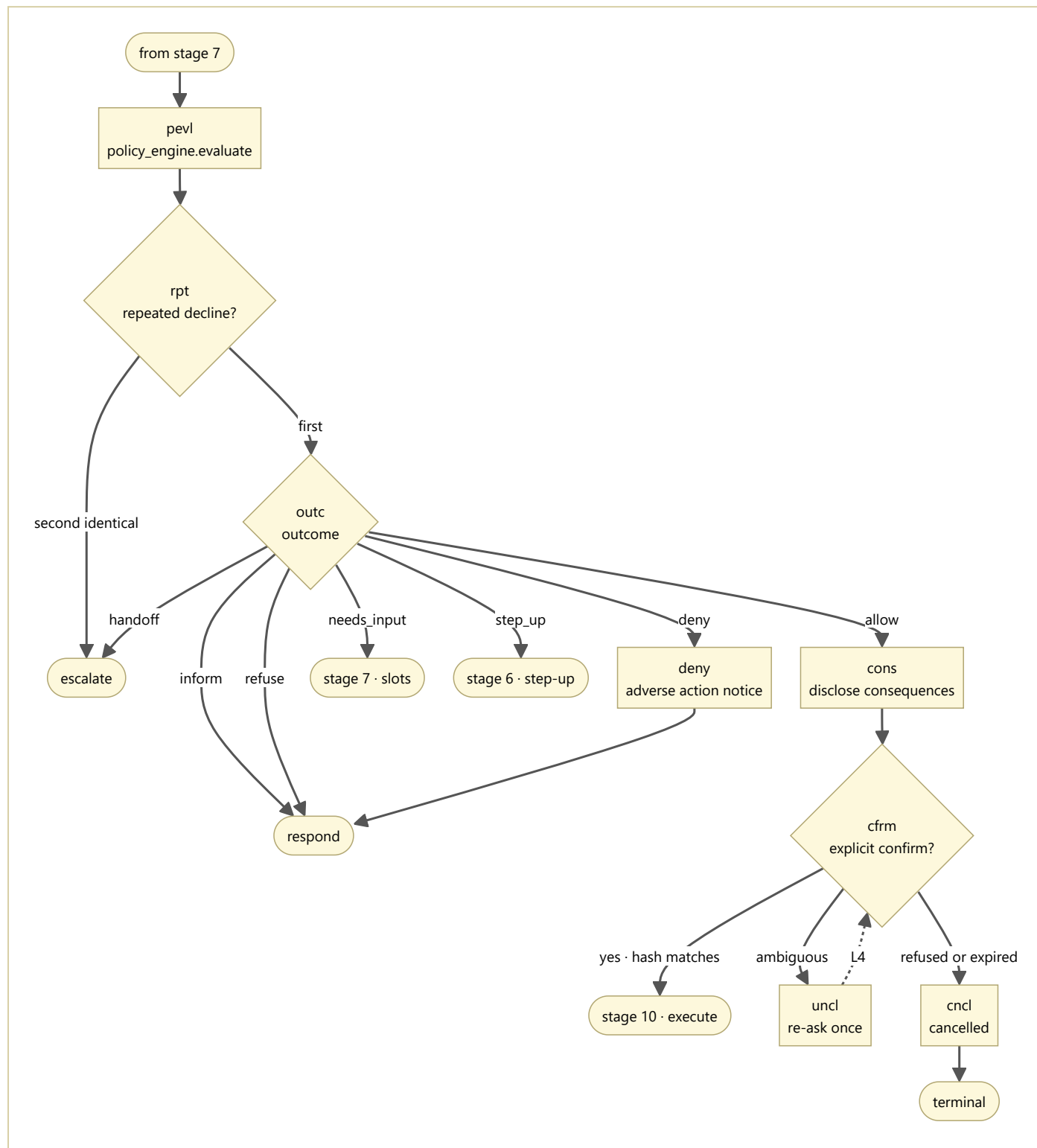
The state profile is chosen before anything is read, and it — not the model, not convenience — decides which fields may be fetched. `fc1s` is the node that makes the whole unbuilt intent catalogue safe today: no approved policy for a consequential request means hand off, never infer approval.



05 Phase D — deciding

stages 8-9

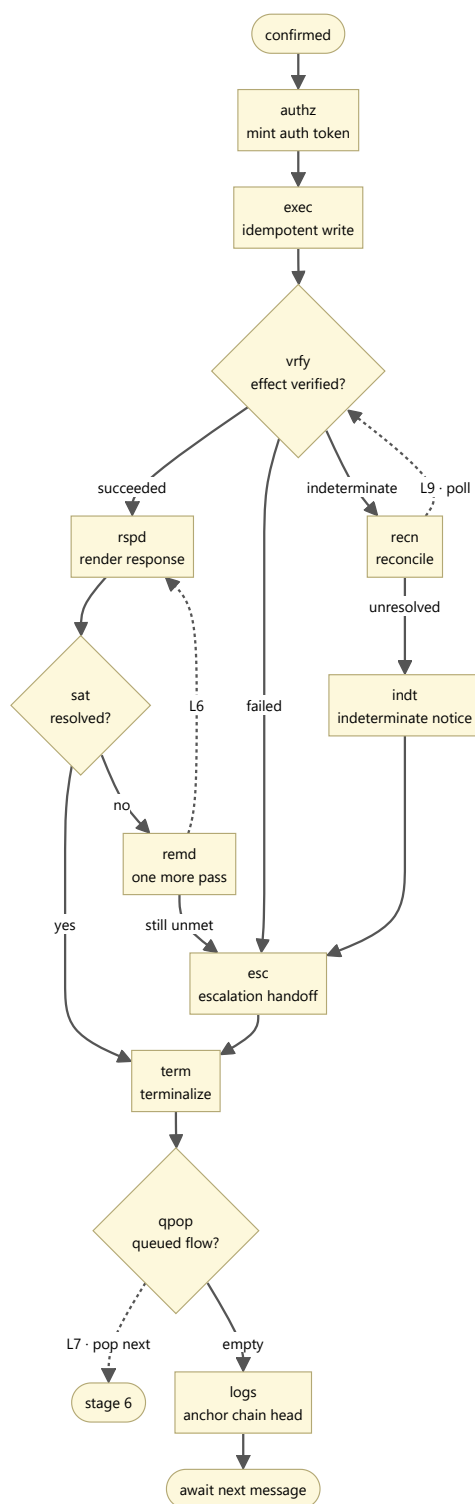
No model is called in this phase. `pevl` is a pure function of account facts and versioned YAML, and its seven outcomes are the only ways out. A repeated identical decline escalates rather than restating itself.



06 Phase E — doing

stages 10-12

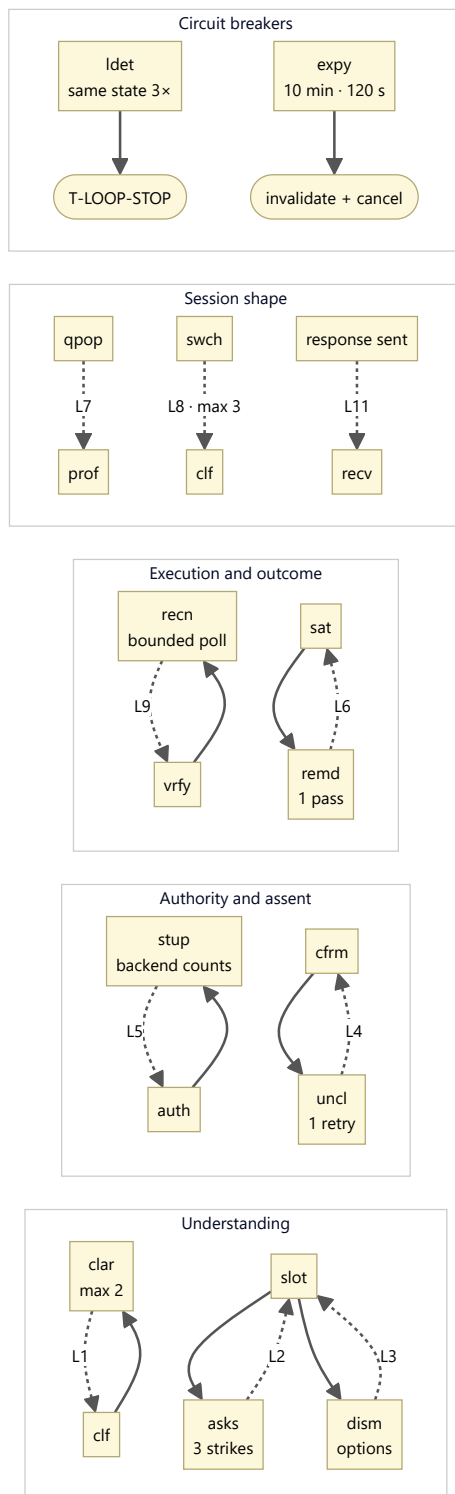
A timeout is *indeterminate* — not failed and not succeeded. The write is reconciled, never retried. That third branch out of `vr-fy` is the difference between an agent that double-charges under load and one that doesn't.



07 Loops

11 cycles

Every loop is bounded and every bound has a named exit. An unbounded loop here is a member trapped in a conversation with no human on the other end.



ID	LOOP	ENTERED WHEN	BOUND	ON BREAK
L1	Intent clarification	Confidence 0.55–0.82, or margin under 0.12	2 questions	T-INTENT-UNRESOLVED → escalate
L2	Slot fill	A required slot is missing or invalid	3 attempts, escalating templates	Offer human or cancel; nothing executed

ID	LOOP	ENTERED WHEN	BOUND	ON BREAK
L3	Disambiguation	Two or more candidate targets match	Candidate count; folds into L2	Bounded choices, then L2's third strike
L4	Confirmation	Assent is ambiguous rather than explicit	1 retry, 120 s TTL	T-CANCELLED, action never submitted
L5	Step-up auth	Operation needs higher assurance	Owned by the auth backend, never counted locally	T-AUTH-LOCKED → escalate
L6	Satisfaction remediation	Member says the outcome did not resolve the need	1 pass	Escalate with full attempt history
L7	Queued flow	Multi-intent turn, or deferred topic switch	Queue depth	Offer handoff; never execute an abandoned confirmation
L8	Topic switch	Member changes subject mid-flow	3 unresolved switches	Offer handoff, preserve the queue
L9	Reconciliation	The write returned indeterminate	Bounded poll window	T-INDETERMINATE → escalate. Never retried.
L10	Loop detector	Same state and question 3× with no new information	3 repeats	T-LOOP-STOP → escalate
L11	Turn	Any non-terminal response was sent	10-minute idle expiry	Invalidate auth; require re-auth to resume

THE RULE THAT KEEPS THE LOOPS HONEST

L2, L4 and L5 all touch attempt counting, and only one of them may count. Slot and confirmation retries are conversation state and belong to the agent. Authentication and OTP attempts belong to the auth backend — the conversation layer displays its status and never independently counts a secret-entry attempt.

08 Stack assignment

The listed stack contains three redundant pairs. Each component gets one job it is uniquely good at — and where there is no distinct job, that is stated.

COMPONENT	JOB IN THIS SYSTEM	CALL
React / Next.js	Member chat, audit viewer, agent console	deferred
Node.js	BFF: SSE streaming, session cookie, rate limiting, channel adapters. No decision logic.	keep
FastAPI	Graph orchestration, policy engine, tool layer, autonomy-tier enforcement	keep
Rasa	Intent, entity and dialogue-act classifier. Self-hosted, so messages never leave the VPC for routine classification.	keep
GPT-4 API	Language only: paraphrase inside an approved template, phrase the clarify question. Never sees a threshold.	keep
Dialogflow CX	Duplicates Rasa. The only non-overlapping role is voice or IVR ingress.	cut / voice-only
Splunk	Audit index via HEC, decision-history retrieval, policy binding lookup, quality-gate dashboards	primary
Elasticsearch	kNN semantic search over Amex India public source documents — the one retrieval Splunk cannot do	narrow role
PostgreSQL	System of record: accounts, transactions, the hash chain, idempotency keys	keep
MongoDB	Live flow state — a nested document, not a relational row	keep
AWS	ECS Fargate, RDS, Secrets Manager, KMS for the chain key, S3 Object Lock for the chain-head anchor	primary
GCP	Carries Dialogflow CX and nothing else	conditional

09 Retrieval and audit

"Retrieval" is one box in the sketch but five different reads against four stores. Splunk owns two of them, and they are the two it is genuinely built for.

ID	WHAT IS READ	STORE	WHY THIS STORE
R1	Live flow state — slots, status, risk flags, counters	MongoDB	Deeply nested and rewritten every turn; a document, not a row
R2	Decision history — prior declines, repeat detection	Splunk	Time-ordered event search over an append-only index is exactly Splunk's shape
R3	Policy binding — intent to policy id, version, tier	Splunk KV Store	Small, structured, keyed. Semantic search would be the wrong tool.
R4	Amex India public source text backing the answer	Elasticsearch	Prose needs semantic matching. This is the retrieval Splunk cannot do.
R5	Account facts — limit, tenure, delinquency, card status	PostgreSQL	Transactional truth; the same connection later carries the write

CONSISTENCY BOUNDARY

R2 reads Splunk, which is eventually consistent behind HEC ingest. Repeat-decline detection must tolerate a lag of seconds. If a decision needs a strictly consistent read of prior decisions, take it from the Postgres chain and treat Splunk as the search surface over the same events.

On the write side, Postgres holds the hash chain because `prev_hash` needs a serialized writer; Splunk holds the searchable copy because that is where dashboards and quality gates live.

AUDIT FIRES	WHAT IS RECORDED	EMITTED AT
Every turn	Redacted message and its hash, dialogue act, intent candidates with confidence, margin and evidence spans, modifiers, risk flags, state version and prior-state hash, template id and rendered text	recv, dact, clf, rspd
Every policy evaluation	Policy id, version, source tag and reference, redacted inputs checked, decision, reason code, rule fired	pevl
Every tool call	Tool name and version, redacted parameters, token references, idempotency key, request and response hashes, status, latency, read-back	authz, exec, vrfy
Every security event	Detection class, redaction applied — never the secret value	sec, abu, ldet
Terminal	Outcome, receipt or case reference, unresolved items, handoff packet reference, chain head anchor	term, logs

WHY THIS IS THE CHANGE THAT MATTERS

A ledger that only records successful writes cannot audit a decline — and a decline is where a member is most likely to challenge the outcome. Emitting on every transition, not only on execution, is what turns the trail from a record of actions into a record of decisions.

10 Open decisions

- **Dialogflow CX.** It duplicates Rasa. Keep it only if a voice channel is in scope; cutting it also removes the GCP dependency and leaves a single-cloud deployment.

- **Satisfaction check.** Asking every member "did that resolve it?" is the honest way to measure first-contact resolution and mild friction on a clean approval. Consider asking only after a decline, a queue or a partial resolution.
- **Effect verification against a local database.** The read-back is nearly tautological when write and read hit the same transaction. Worth building for the interface it establishes, but the indeterminate branch is only genuinely exercised against a remote core.
- **Rasa versus the existing trained classifier.** There is already a working embeddings-and-logistic-regression classifier at 94.9% held-out. Rasa DIET is the better production answer; the switch costs a retrain and a fresh held-out evaluation, so sequence it after the audit work.

Full node inventory with owner-per-node lives in DECISION_GRAPH.md in the repo.